POO C#

•••

GT3 - Kévin Roussel

Rappel

- Classe : rassemble des champs et des méthodes pour répondre à un besoin spécifique
- Champs : information préservée au cours du jeu.
- Méthodes : fonctionnalité de la classe, peut être ouvert à tous (public) ou uniquement disponible dans la classe (private)

- Scope : porté, une variable créée dans un scope est disponible uniquement dans ce dernier.

Conditions

```
if(CONDITION)
{
}
else if(CONDITION)
{
}
else
{
}
```

Opérateur	Description
==	Egalité
!=	Différence
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal
&&	ET logique
||	OU logique
!	Négation

Instruction - switch

Permet de tester un ensemble de valeur possible pour une information donnée.

Souvent utilisé avec des enum

case

default

break;

```
switch(state)
{
        case PATROL :
                Debug.Log("Patrouillons");
                break;
        case CHASE:
                Debug.Log("Tiens vla quelqu'un");
                break;
        case ATTACK:
                Debug.Log("A l'assault !");
                break;
        default :
                Debug.Log("Je sais pas ce que je fais");
                break;

}
```

Boucles : while

On refait un ensemble d'opération TANT QUE la condition donné est vrai.

Attention, si cette condition est fausse dès la première fois on saute le bloc

```
while ( CONDITION)
{
        Debug.Log("C'est beau mais c'est loin");
}
```

Boucles : do while

Même chose que le while à la seule différence que l'on évalue la condition à la fin du bloc de code. On est donc assuré de faire le bloc au moins une fois.

```
do
{
        Debug.Log("C'est beau mais c'est loin");
} while ( CONDITION);
```

Boucles : for

Parfois on veut faire une boucle un certain nombre de fois déjà défini à l'avance.

Le for nous permet de rassembler ces instructions en une ligne. Utile pour parcourir un tableau

```
for( INIT ; CONDITION ; ITERATION )
{

}
```

Boucle : foreach

Évolution du for, permet de parcourir de bout en bout une collection (IEnumerable)

```
public List<string> jours = new List<string>() { “lundi”, “mardi”, ... };
Debug.Log(“le facteur n’est pas passé, il ne passera jamais”);
foreach ( string j in jours )
{
    Debug.Log(j);
}
```

Si une méthode ne possède qu'une seule ligne de code

On peut utiliser une syntaxe permettant de tout faire sur une seule ligne

```
void SayHelloWorld() => Debug.Log("Hello World");
```

vs

```
void SayHelloWorld()
{
    Debug.Log("Hello World");
}
```

Propriétés

Habituellement ...

Nos champs sont privés, donc il est de coutume de créer une méthode qui se contente de nous retourner la valeur d'un champs. De cette manière l'information peut être disponible.

```
string _myMessage;

string GetMyMessage() => _myMessage;
```

C’est la qu’intervient les propriétés

“Syntaxique Sugar” permettant de simplifier l’écriture de ces méthodes basiques

```
string _myMessage;
public string MyMessage
{
        get => _myMessage;
        set => _myMessage = value;
}
```

On peut customiser la visibilité

```
public string Message
 {
        get => _message;
        private set => _message = value;
}
```

On peut également omettre le corps du getter et du setter

```
public int CurrentHealth { get; private set; }
```

(Ici le compilateur nous crée automatique un champs qu'il utilise dans la propriété)

Résumé

Dès que l'on veut rendre accessible une information au reste du monde, on expose alors une propriété.

De cette manière, on garde le contrôle sur nos champs. On peut proposer aux autres composants de changer les valeurs MAIS ils doivent passer par nous (set), ce qui nous permet de nous assurer qu'on ne nous injecte pas des valeurs bidons.

Events

A quoi ça sert

Déclarer qu'il s'est passé quelque chose de spécifique. Les composants qui se sont inscrit à cet évènement va voir sa fonction enregistrée appellée pile au moment de l'invocation de l'event.

Une fonction est assignable à un objet

Une fonction retournant void :

Action

Une fonction void avec un premier paramètre int :

Action<int>

Une fonction void avec un paramètre int puis string :

Action<int,string>

Et ainsi de suite...

Une fonction est assignable à un objet

Une fonction retournant int :

Func<int>

Une fonction retournant int avec un param string :

Func<string, int>

On enchaîne les types des params puis en dernier on indique le type de retour

Déclaration d'un event

Dans notre classe Health, lorsque le personnage meurt on veut un event OnDie.
On n'a pas spécialement d'information à fournir à ce moment donc on utilisera juste le type Action

```
public event Action OnDie;
```

Pour le déclencher :

```
void Damage(int amount)
{
    _hp -= amount;
    if(_hp<=0)
    {
        OnDie?.Invoke();
    }
}
```

La subtilité ici c'est que l'event est potentiellement *null* si personne ne s'y est inscrit.

Pour régler ça on utilise traditionnellement l'opérateur ?.

Pour s’inscrire à un event :

```
Health h = new();

h.OnDie += MaReactionOnDie;

// Un peu plus loin j'ai une fonction qui est là pour gérer la mort
void MaReactionOnDie()
{
    Console.Write("Olala je suis mort")
}
```

La désinscription est logique du coup

```
h.OnDie -= MaReactionOnDie;
```

C’est important de se désinscrire pour nettoyer la mémoire auprès de ceux dont on s’est inscrit. ça va notamment être primordiale lors de la gestion du cycle de vie d’objets plus complexe.

On aura une utilisation des events bien plus importantes plus tard

Notamment sur des systèmes qui soient de plus en plus complexe.

On peut penser par exemple à un contexte de gameplay où l'on veut éviter que des objets spam de demande un autre objet.

Exemple : une barre de vie qui veut se refresh à chaque frame. Aucun intérêt, il est plus pertinent que l'UI se branche sur un event pour réagir lorsque la vie du perso change : On diminue drastiquement le nombre d'appel.

Donc on a :

Des champs, nos informations qui persiste dans le temps

Des propriétés, pour rendre disponible nos informations

Des méthodes, pour faire du travail concret

Des évènement pour avertir le monde d'une situation donnée

Static

Des nouvelles subtilités

Le modificateur static : le membre concerné est écrit dans le type et non dans les instances, de cette manière on peut avoir une information en commun.

Une méthode public static est dispo à tous et peut être appelée à n'importe quel moment : Debug, Mathf, JSONUtility, PlayerPrefs ...

... Et donc Console également !

Utilisation ultra classique : Singleton

```
public class GameManager
{

    public static GameManager Instance;


}
```

Structure

Maintenant les structures ...

- Plus contraignant
- ... Mais aussi bien plus optimisé
- Utile pour de petites informations
  - Vector3
  - Color
  - RaycastHit

new

Quand on veut créer un nouveau Vector3, il faut utiliser le mot-clé new

```
var direction = new Vector3(1 , 2 , 3);
```

Constructeur

De l’autre côté de la barrière

```
public struct Vector3
{
    float x, y, z;
    public Vector3 (float myX, float myY, float myZ)
    {
        x = myX; // etc
    }
}
```

Recréons Vector3

- champs _x, _y, _z
- Propriété X, Y, Z
- propriété Magnitude
- propriété SqrtMagnitude
- méthode Normalize
- surcharge d'opérateur +

Et faites les tests unitaires qui pourrait prouver que votre travail fonctionne